

Introduction to Databases

G-classroom => of5lsbaf

Assoc. Prof. Kitsana Waiyamai, Ph. D.
Dept. Of Computer Engineering
Faculty of Engineerring, Kasetsart University
Bangkok, Thailand

Slides based on Silberschatz et Al.



Objective of Our Database Class

- **Understand key database concepts and their importance**
  - Key database concepts.
  - Role and importance of databases in various applications.
- **Develop Skills in Database Design & Implementation**
  - Create efficient database schemas.
  - Basic SQL and
  - Complex SQL for querying and manipulating databases.
- **Identify and Compare Different Types of Database Models**
  - Explore various database models
    - **Relational Model**, **No-SQL Models**
    - **Modern Specialized Models**.
  - Compare their advantages and limitations.
- **Select appropriate database models for different scenarios**
  - Analyze different problem scenarios and select the most suitable database model.
  - Understand the criteria for choosing between **Relational**, **NoSQL**, and other **Models**.
- **Practical Application**:
  - Solve real-world problems through projects and case studies.
- **Explore Modern DB Trends**: Vector DB, Cloud DB, …
- **Collaboration**:
  - Enhance teamwork and communication skills through group projects.

Introduction to Databases



- Data VS Information VS Knowledge
- What is Database?
- Different types of Database Models
- Conceptual Schema VS Logical Schema
- DBMS and Its Components



Various Types of Data

XML

```
<Computer>BAC283</Computer>
</System>
<EventData>
 <Data Name="ContextId">{80000396-0000-F200-B63F-8
 </Data>
 <Data Name="HeaderName">AspFilterSessionId</Data>
 <Data Name="HeaderValue"></Data>
 <Data Name="Replace">true</Data>
</EventData>
<RenderingInfo Culture="th-TH">
 <Opcode>GENERAL_SET_REQUEST_HEADER</Opcode>
</RenderingInfo>
<ExtendedTracingInfo xmlns="
http://schemas.microsoft.com/win/2004/08/events/tr
 <EventGuid>{D42CF7EF-DE92-473E-8B6C-621EA663113A}
</ExtendedTracingInfo>
</Event>
<Event xmlns="
http://schemas.microsoft.com/win/2004/08/events/eve
 <System>
  <Provider Name="WWW Server" Guid=
  "{3A2A4EB4-4C21-4981-AE10-3FDA0D9B0F83}"/>
  <EventID>0</EventID>
  <Version>1</Version>
  <Level>5</Level>
  <Opcode>5</Opcode>
  <Keywords>0x100</Keywords>
  <TimeCreated SystemTime="2016-10-17T07:16:12.136Z
  <Correlation ActivityID="{80000396-0000-F200-B63F
```

JSON

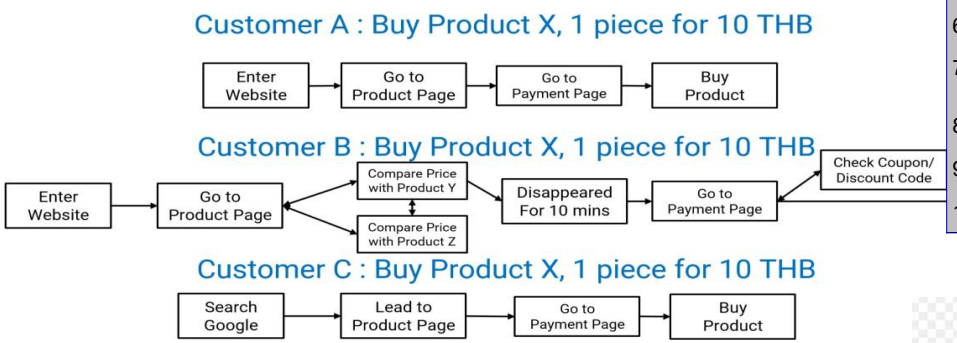
```
}, {
  "timestampMs" : "1491569685424",
  "latitudeE7" : 137531234,
  "longitudeE7" : 1005340501,
  "accuracy" : 50
}, {
  "timestampMs" : "1491569571536",
  "latitudeE7" : 137531234,
  "longitudeE7" : 1005340501,
  "accuracy" : 50
}, {
  "timestampMs" : "1491569372905",
  "latitudeE7" : 137531234,
  "longitudeE7" : 1005340501,
  "accuracy" : 50
}, {
  "timestampMs" : "1491569311365",
  "latitudeE7" : 137531234,
  "longitudeE7" : 1005340501,
  "accuracy" : 50,
  "activity" : [ {
    "timestampMs" : "1491569311649",
    "activity" : [ {
      "type" : "STILL",
      "confidence" : 100
    } ]
  } ]
}, {
```

Complex LOG

```
192.168.2.22:9300                Default Executor-thread-6   LOGSV   6100    1   server.Au
192.168.2.22:9300                Default Executor-thread-6   LOGSV   6100    1   server.Au
192.168.2.22:9300  Initialization_SESS Initialization_REQ          Thread-35   CH  6100    1
192.168.2.22:9300                Thread-30   LOGSV   6100    1   server.Audit.IPF    Execu
192.168.2.22:9300                Thread-35   CH  6100    1   Audit.RTUsage.cms.CH    Start
192.168.2.22:9300  Initialization_SESS Initialization_REQ          Thread-35   CH  6100    1
192.168.2.22:9300  Initialization_SESS Initialization_REQ          Thread-35   CH  6100    1
192.168.2.22:9300                Thread-35   CH  6100    1   Audit.RTUsage.cms.CH    Start
192.168.2.22:9300  Initialization_SESS Initialization_REQ          Thread-35   CH  6100    1
192.168.2.22:9300  Initialization_SESS Initialization_REQ          Thread-35   CH  6100    1
192.168.2.22:9300  Initialization_SESS Initialization_REQ          Thread-35   CH  6100    1
192.168.2.22:9300  raProcessStarter    na      0   CAMLPS-ProcessMgrThread DISP    6100    1
192.168.2.22:9300  BootstrapConfigurePublish   na      0   Thread-35   DISP    6100    1   A
192.168.2.22:9300                Timer-28    ATHENA  5000    1   Audit.RTUsage.idViz.ATHEN
192.168.2.22:9300                Timer-28    AAA 6100    1   Audit.cms.CAM.AAA   StartServ
192.168.2.22:9300                Timer-28    AAA 6100    1   Audit.cms.CAM.AAA   StartServ
192.168.2.22:9300  na  na        Timer-28    DISP    6100    1   Audit.Other.dispatcher.DI
192.168.2.22:9300  na  na        Timer-28    DISP    6100    1   Audit.Other.dispatcher.DI
192.168.2.22:9300  na  na        Timer-28    DISP    6100    1   Audit.Other.dispatcher.DI
192.168.2.22:9300  na  na        Timer-28    DISP    6100    1   Audit.Other.dispatcher.DI
192.168.2.22:9300  na  na        Timer-28    DISP    6100    1   Audit.Other.dispatcher.DI
192.168.2.22:9300  na  na        Timer-28    DISP    6100    1   Audit.Other.dispatcher.DI
192.168.2.22:9300  na  na        Timer-28    DISP    6100    1   Audit.Other.dispatcher.DI
192.168.2.22:9300  na  na        Timer-28    DISP    6100    1   Audit.Other.dispatcher.DI
TRU-MIE-OPR-AUX-RMAIN  GRAD-10019  DM  Breakdown  2017-01-16 13:25:43  WO17100648  CLOSE  ICHAMADIN_A  GD009 HOSE CYLINDER LIFT LEAK HM:13656
TRU-MIE-OPR-AUX-BOOZ   DOZE-10036  DM  Breakdown  2017-01-16 13:26:45  WO17100649  CLOSE  ICHAMADIN_A  BD020 RESEAL OIL COOLER HM:20537   ACH
TRU-MIE-AM-EQUIPT-TFUEL GENS-10225 DM  Breakdown  2017-01-16 13:28:03  WO17100650  CLOSE  GUNAWAN_GUNAWAN DT060 FRONT SPRING RH BROKEN   ACHMADI
TRU-MIE-OPR-OHT-HD465  TRCK-10560  DM  Breakdown  2017-01-16 13:30:08  WO17100651  CLOSE  MAHADE_K   TR057 TIRE FS LEAKS ACHMADINATA_A  2017-01
TRU-MIE-OPR-EXCAV-MED  EXCA-10053  DM  Breakdown  2017-01-16 13:31:10  WO17100652  CLOSE  CHRISTANTO_C  EX018 HOSE SWING MOTOR FRONT LEAKAGE hm
TRU-MIE-OPR-OHT-HD785  TRCK-10549  DM  Breakdown  2017-01-16 13:32:34  WO17100653  CLOSE  MAHADE_K   TR029 HOSE DUMP - LH LEAKAGE HM:22229   ACH
TRU-MIE-OPR-EXCAV-MED  EXCA-10047  DM  Breakdown  2017-01-16 13:33:33  WO17100654  CLOSE  CHRISTANTO_C  EX022 PIN JOINT ARM NOSE CYLINDER BROKE
TRU-MIE-OPR-AUX-DRILL  DRIL-10464  DM  Breakdown  2017-01-16 13:34:44  WO17100655  CLOSE  ICHAMADIN_A  RD002 HOSE DRAIN AIR COMPRESSORE LEAKAG
TRU-MIE-OPR-EXCAV-MED  EXCA-10053  DM  Breakdown  2017-01-16 13:35:46  WO17100656  CLOSE  CHRISTANTO_C  EX018 HOSE DELIVERY MAIN PUMP LEAKAGE
TRU-MIE-OPR-EXCAV-LARGE EXCA-10056 DM  Breakdown  2017-01-16 13:36:46  WO17100657  CLOSE  CHRISTANTO_C  EX008 AIR CONDITIONER HOT   ACHMADINATA
TRU-MIE-OPR-OHT-HD465  TRCK-10528  DM  Breakdown  2017-01-16 13:39:16  WO17100659  CLOSE  MAHADE_K   TR074 NO START  ACHMADINATA_A  2017-01-16
TRU-MIE-OPR-AUX-BOOZ   DOZE-10055  DM  Breakdown  2017-01-16 13:40:21  WO17100660  CLOSE  ICHAMADIN_A  BD019 ROTARY LAMP OFF   ACHMADINATA_A
TRU-MIE-OPR-EQUIPT-TANKO  TRCK-10527  DM  Breakdown  2017-01-16 13:41:59  WO17100661  CLOSE  GUNAWAN_GUNAWAN DT054 HOSE CHARGING LEAKAGE HM:7033
```

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes









+ Other Complex Types of Data

Data vs Information vs Knowledge

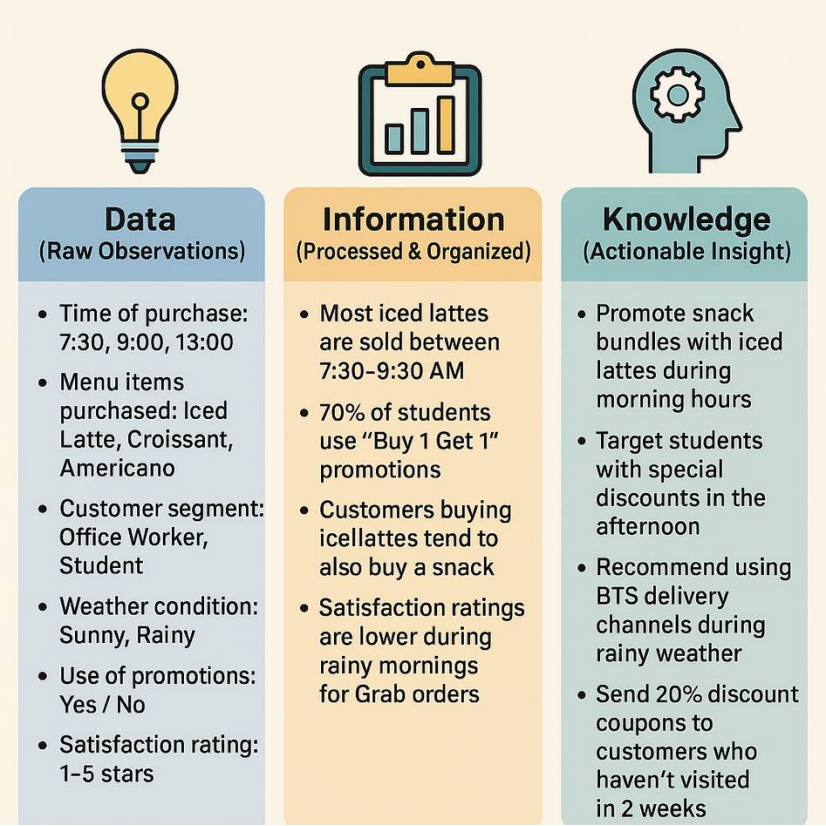
- Data: Collection of facts, such as numbers, words, measurements, observations or just descriptions of things.
- Information: Data that has been *processed* into a form that is meaningful to the recipient.
- Knowledge: Combination of ideas, rules, and procedures that guide *actions* and *decisions*.





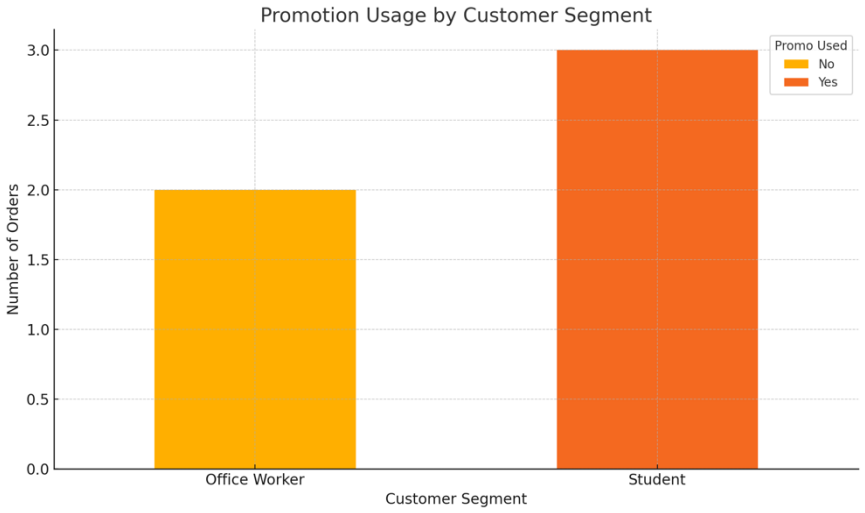
Data-driven Decision

CoffeeConnect
Case



Time	Item Purchased	Segment	Weather	Used Promo	Rating
07:30	Iced Latte, Croissant	Student	Sunny	Yes	5
09:00	Iced Latte	Office Worker	Rainy	No	3
13:00	Americano, Croissant	Student	Sunny	Yes	4
08:45	Iced Latte	Student	Rainy	Yes	2
13:00	Iced Latte, Americano	Office Worker	Rainy	No	4

Data



Information

Data-driven decision can lead to better outcomes in even simple scenarios

What is Database?

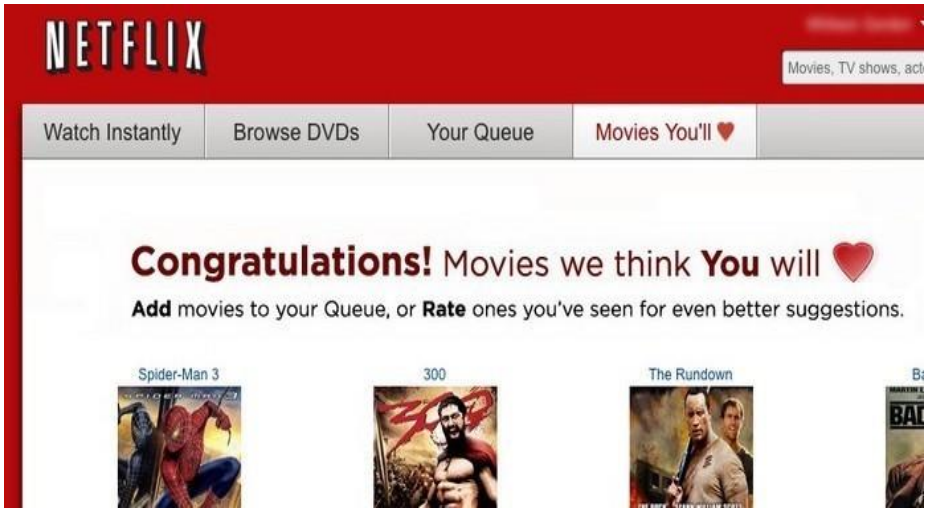
- **Database**: “A collection of interrelated data”
- **Example**:
  - **University** database =>
  - Collection of
    - students, instructors, courses, registrations and grading.
  - **Music** database =>
  - Collection of
    - artists, their recordings,
    - concerts, albums, etc.
- Organized, Structured collection of data stored electronically
- Enables easy access, management, modification, and updating

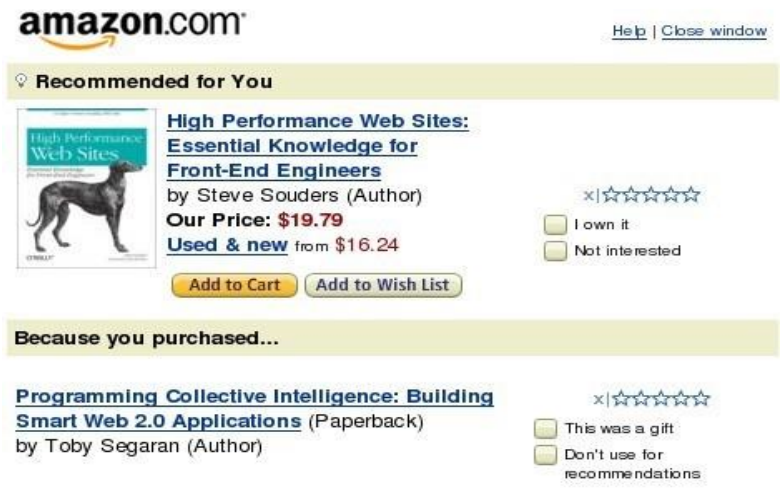




Databases touch all aspects of our lives

- You're using a database every day
  - **Ride-Hailing Apps:** Tracking driver-locations, trip history
  - **Streaming Platforms:** Storing user profiles, watch histories, feed ranking, movies recommendations
  - **E-Commerce & Banking:** Order tracking, Managing inventories, account balances, and secure transactions
- Some apps use very Huge Databases
  - **Facebook / Meta** – social graph, photos, interactions
  - **YouTube** – video metadata + logs
  - **Netflix** – recommendations, viewing history, global metadata
  - **TikTok** – real-time engagement + videos
  - **Amazon / Shopee** – orders, products, payments
  - **Grab / Uber** – GPS updates, trips, payments
  - **Twitter/X** – tweets, followers graph
  - **Spotify** – playlists, user activity
  - **Google Maps** – global geospatial + traffic
  - **Alibaba** – high-volume transactions and analytics
  - Etc.

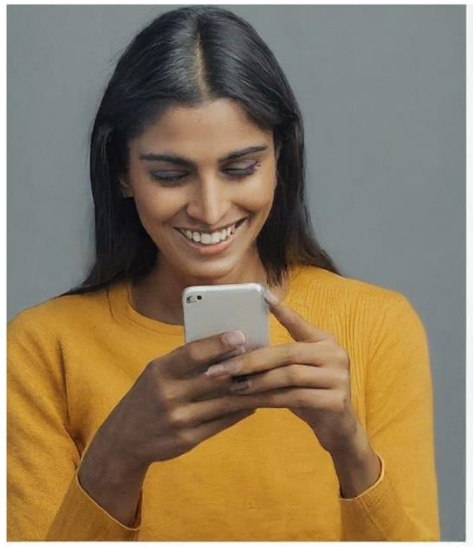




Benefits of Databases



- **AI Needs Databases**
  - **Data Pipeline:** AI models require clean, structured data for training
  - **Retrieval Augmented Generation (RAG):** Contextual retrieval of information for LLMs, Storing chat history, embeddings
  - **Supporting predictive analytics and machine learning models.**
    - Storage and management of large datasets for training models.
    - Enhanced predictive capabilities leading to Personalization
    - Proactive decision-making.
  - **Enabling generative AI applications.**
    - Efficient storage and access to training data for generative models.
    - Support for large-scale data required by models like GPT-4.
- **Centralized repository for data storage.**
  - Organized and structured data.
  - Secure and scalable storage solutions.
  - Data integrity and Consistency.
- **Efficient querying and manipulation of data.**
  - Rapid access to relevant information.
  - Support for complex queries, joins, and aggregations.
- **Real-Time Processing:**
  - Support for real-time applications, such as monitoring and alerting.
  - Improved user experience through instant data processing and feedback.
- **Integrating data from multiple sources into a unified database.**
  - Comprehensive view of data from various systems.
  - Improved data consistency and accuracy.
- **Etc**.





Other Example of Databases

- **Databases Examples**
  - **Universities**: students, courses, registration, grades
    - Add new students, instructors, and courses
    - Register students for courses
    - Assign grades to students, compute grade point averages (GPA) and generate transcripts
  - **Banking**: customers, accounts, transactions
  - **Airlines**: passengers, reservations, schedules
  - **Sales**: customers, products, purchases
  - **Entertainment**: Video- streaming
  - **Online retailers**: order tracking, customized recommendations
  - **Manufacturing**: production, inventory, orders, supply chain
  - **Human resources**: employee records, salaries, tax deductions
- **Almost all domains !**

Database and its related terms

Database: “A collection of related data represented (using database model + database schema)”

Database Model: “A formal definition on how to represent data and the available data operations.”

Database Schema: “A definition of the structure of a specific database”

Database Management System (DBMS): “A software system managing and maintaining databases”

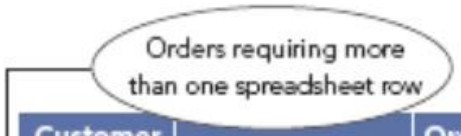
Database Models

- Database Model:
  - A formal definition on how to represent data and the available data operations.
  - It serves as a blueprint for how data is stored, accessed, and manipulated.
  - Data models help ensure data consistency, integrity, and efficient management by providing a systematic approach to organizing data.
- Evolution of Database Model & Technology
  - < 1980: **Old models**: Object-based models, Object-relational model, Network model, Hierarchical model, …
  - 1980: **Relational models**, Semi-structured model, for example XML
  - 2000: **NoSQL models**:
    - Document model, Graph model, Key-value model, … .
  - 2010: **Vector DBs**, **Cloud DBs**
  - 2020+: **Multi-models**, Vector DBs, Real-time Analytics, Serverless

Single-Table Model

- Single-Table Model: Represents all data as a Single Table
- Typically:
  - This table can be easily represented by a CSV, EXCEL, XML files
  - No DBMS necessary to manage those file (there are however DBMS specialized on this)
  - File can be / must be directly manipulated by humans or software
  - Lack of standardized DBMS make querying and handling hard
- Data file using this model files are mostly just used for exchanging data

Orders Stored in Spreadsheet



Customer Number	Customer Name	Order Number	Order Date	Part Number	Part Description	Number Ordered	Quoted Price	Warehouse	Rep Number
148	Al's Appliance and Sport	21608	10/20/2010	AT94	Iron	11	$21.95	3	20
148	Al's Appliance and Sport	21619	10/23/2010	DR93	Gas Range	1	$495.00	2	20
282	Brookings Direct	21614	10/21/2010	KT03	Dishwasher	2	$595.00	3	35
356	Ferguson's	21610	10/20/2010	DR93	Gas Range	1	$495.00	2	65
356	Ferguson's	21610	10/20/2010	DW11	Washer	1	$399.99	3	65
408	The Everything Shop	21613	10/21/2010	KL62	Dryer	4	$329.95	1	35
608	Johnson's Department Store	21617	10/23/2010	BV06	Home Gym	2	$794.95	2	65
608	Johnson's Department Store	21617	10/23/2010	CD52	Microwave Oven	4	$150.00	1	65
608	Johnson's Department Store	21623	10/23/2010	KV29	Treadmill	2	$1,290.00	2	65

Pros and Cons of Single-Table Model

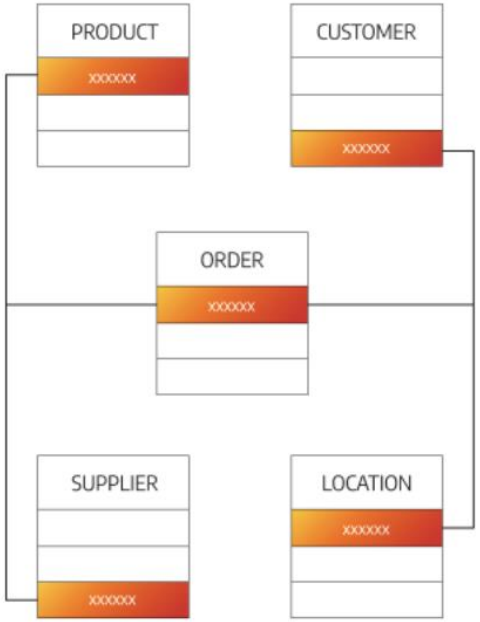
- Pro:
  - Very simple
  - Very effective for reading operations
  - Can easily be converted to files
  - No DBMS necessary
- Con:
  - Inflexible - hard to perform meaningful and complex queries
  - Schema is “flattened” – most conceptual schema information is lost
  - A lot of data redundancy depending on how the schema was flattened

Traditional VS Modern Database Models

- **Relational Databases (RDBMS):**
  - Organized in tables (rows and columns) with strict schemas (e.g., PostgreSQL, MySQL)
- **NoSQL Databases:**
  - Flexible, non-tabular models for unstructured/semi-structured data (e.g., MongoDB, Redis)
- **Vector Databases:**
  - Store and query high-dimensional embeddings for AI and semantic search (e.g., Pinecone, Milvus)
- **Time-Series Databases:**
  - Optimized for timestamped data such as IoT sensors (e.g., InfluxDB)
- **...**

Relational Database Model

- Represents multiple data tables which are linked to each other (via ids and foreign keys)
  - The relational model organizes data into tables (relations) consisting of rows (tuples) and columns (attributes).
  - Each table has a unique primary key to identify rows, and foreign keys to establish relationships between tables.



Relational Database Model

- Represents multiple data tables which are linked to each other (via ids and foreign keys)

Artists Table:

artist_id	name	genre
1	The Beatles	Rock
2	Taylor Swift	Pop

Recordings Table:

recording_id	title	release_date	artist_id
1	Abbey Road	1969-09-26	1
2	Fearless	2008-11-11	2

Songs Table:

song_id	title	length	recording_id
1	Come Together	4:20	1
2	Love Story	3:55	2

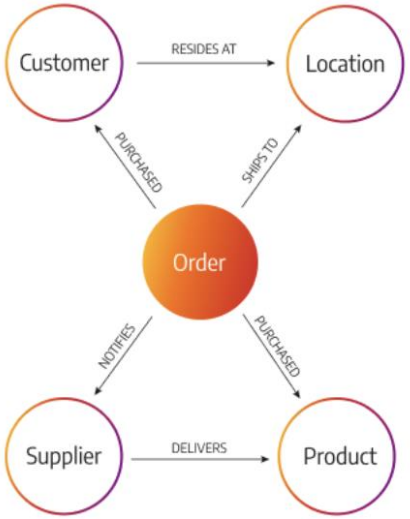
Relational Database Model

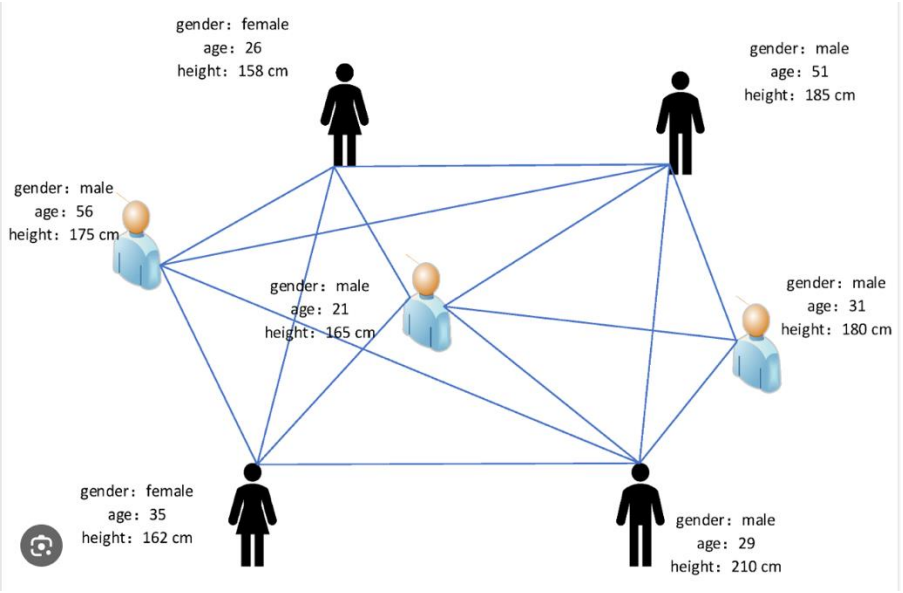
- Represents multiple data tables which are linked to each other (via primary keys ids and foreign keys)

- Typically: Each *entity type* and *relationship type* has its own table
  - A Relational Database Management System (RDBMS) manages those tables and their schemas (like SQL-Server, Oracle, **MySQL**, **PostgreSQL** etc.)
- Powerful SQL Queries can be used to construct any result we
  - The RDBMS makes sure to execute such queries efficiently
- Most large web platforms and enterprise backends use relational DBs

Graph Model

- **Definition:** The graph model uses nodes, edges, and properties to represent and store data. Nodes represent entities, edges represent relationships, and properties store information about nodes and edges.
- **Example:** A social network with nodes representing people and edges representing friendships or interactions.





Key-Value Model

- **Definition:** In the key-value model, data is stored as key-value pairs, where each key is unique and maps to a value.
- **Example:**

Artists:

- Key: `artist:1`, Value: `{name: "The Beatles", genre: "Rock"}`
- Key: `artist:2`, Value: `{name: "Taylor Swift", genre: "Pop"}`

Recordings:

- Key: `recording:1`, Value: `{title: "Abbey Road", release_date: "1969-09-26", artist_id: 1}`
- Key: `recording:2`, Value: `{title: "Fearless", release_date: "2008-11-11", artist_id: 2}`

Songs:

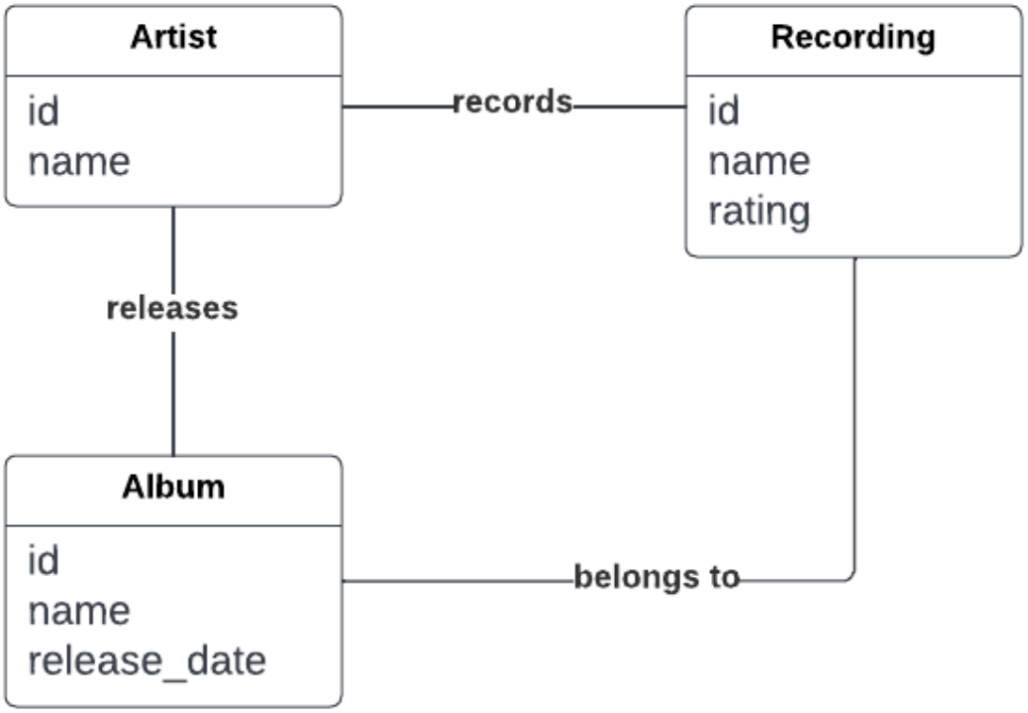
- Key: `song:1`, Value: `{title: "Come Together", length: "4:20", recording_id: 1}`
- Key: `song:2`, Value: `{title: "Love Story", length: "3:55", recording_id: 2}`

Database Schemas

- **Database Schema**: “A definition of the structure of a specific database”
- Two types of schemas: **Conceptual Schema** and **Logical Schema**
- **Conceptual Schema:**
  - High-level view focusing on business **entities** and their **relationships**.
- **Logical Schema:**
  - Implementation of the conceptual schema.
  - Represents all **entity types** and **relationships** to be stored in a database **in the chosen data model.**
  - Detailed structure defining table **attributes**, **data types**, and **constraints**.

Example: UML Conceptual Schema

- **Conceptual Schemas:** Describes all **entity types** and **relationships types** to be stored in a database
- "**What do I want to store?** (and what not?) It's about understanding **what data is needed** and how it's related, without getting into technical details.
- Focuses on the big picture, relationships, and business context.



Conceptual Database Schema

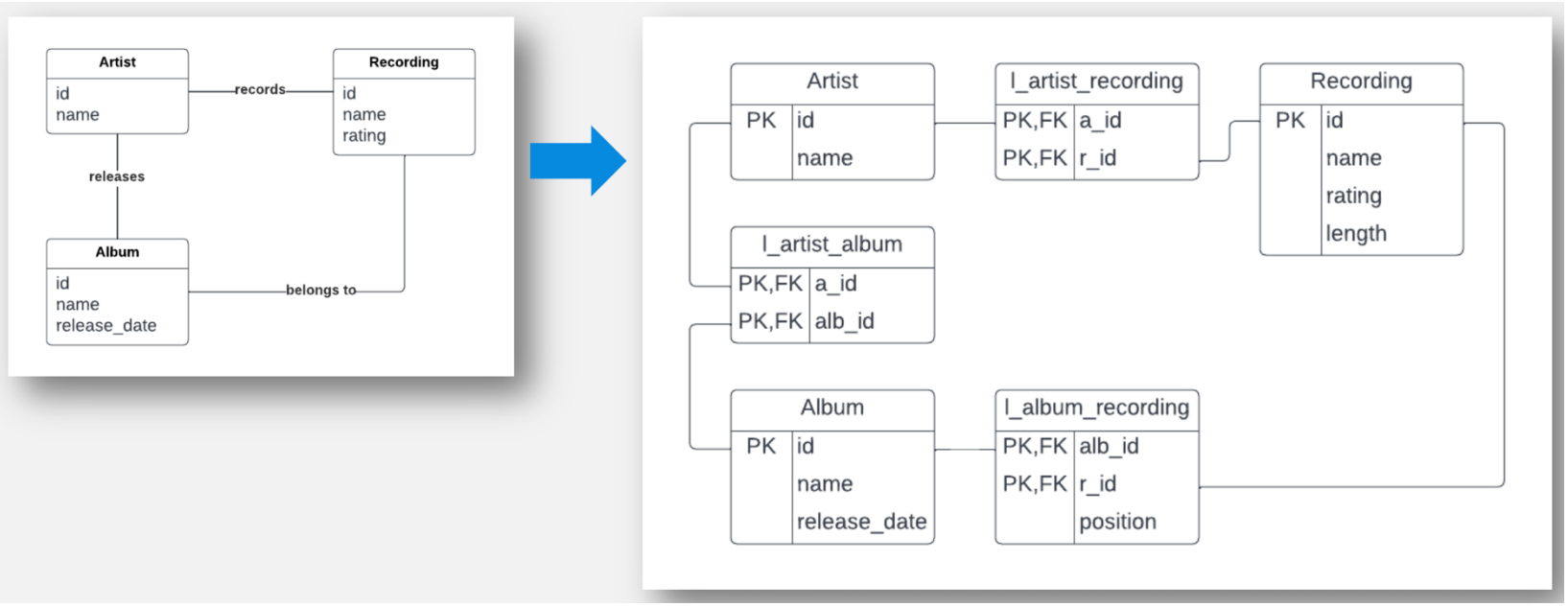
- **Conceptual Schemas:** Describes all **entity types** and **relationships types** to be stored in a database
- **Conceptual Schemas:** Give high-level explanation for users.
- What do I want to store? (and what not?)
- **Example:** UML, ER, …
- **Entity Type**: A type of “thing” which exists in the real world
  - “artist”, “recording”, “album”, etc.
- **Relationships**: “Connections” between entities
  - “an artist *records* a recording”,
  - “a recording is *contained in* an album”,
  - etc.

Logical Database Schema

- **Logical Schemas:** Represents all **entity types** and **relationships** to be stored in a database **in the chosen data model.** Implementation of the conceptual schema.
- **Conceptual Schema** : business point of view
- **Logical Schema** : implementation using a given database model
- **Conceptual Schema** VS **Logical Schema**
  - Conceptual schema is Data Model Independent
  - For the logical schema, we adapt the conceptual schema with the capabilities and restrictions of the Data Model

Example: Conceptual Schema => Logical Schema

- **Logical Schemas:** represents all entity types and relationships to be stored in a database in the chosen data model
- **Example:** Example: A logical schema of our Music database using the relational model

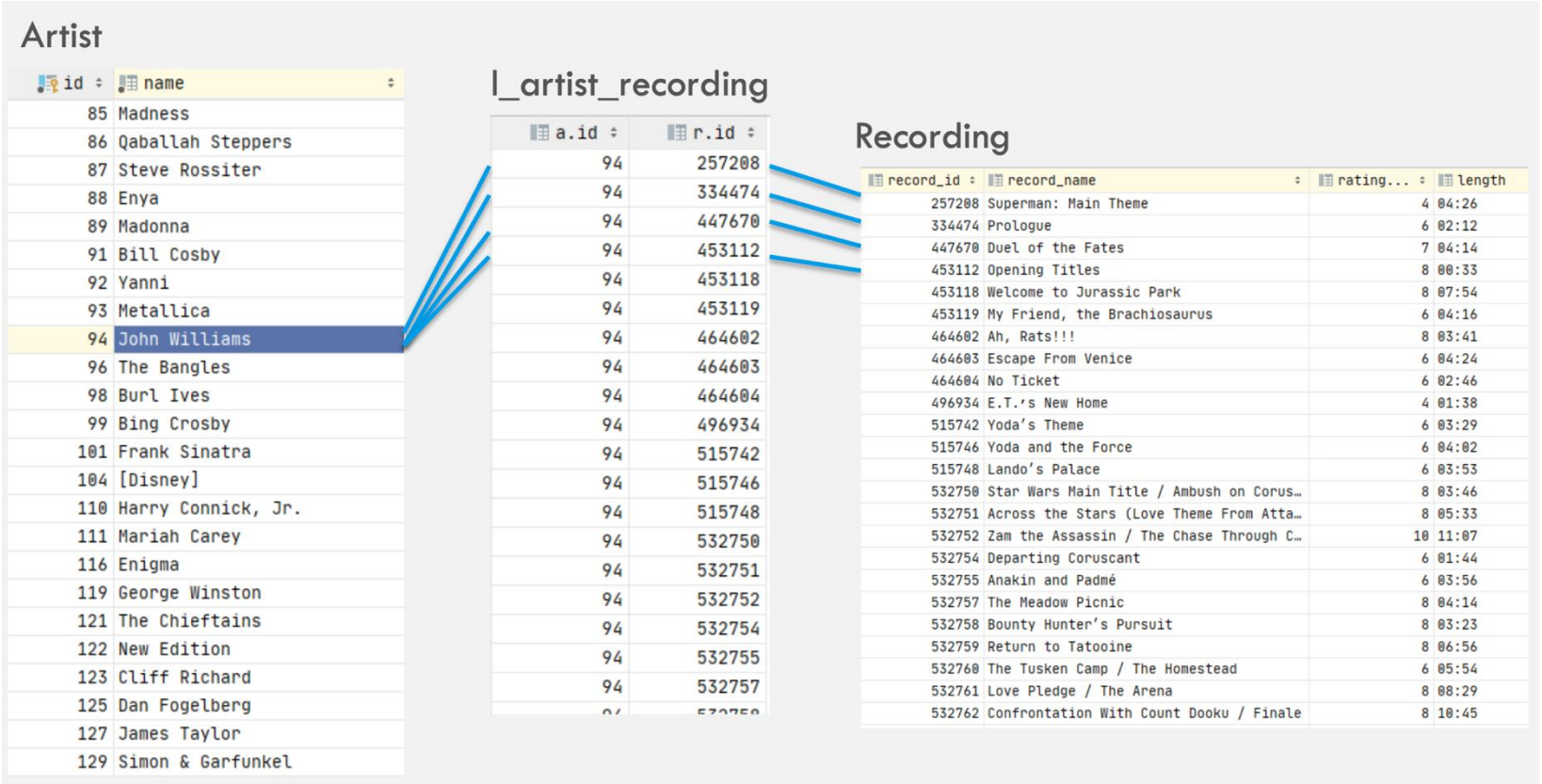


Conceptual Schema

Logical Schema

Example: Logical Schema of Music database

- **Logical Schemas:** represents all entity types and relationships to be stored in a database in the chosen data model
- **Example:** Example: A logical schema of our Music database using the relational model



Example: Logical Schema of College Admission Database using Relational Model

Colleges(cName,state,enrollment)

Students(sID,sName,GPA,sizeHS)

Applies(sID,cName,major,decision)

	sID	sName	GPA	sizeHS
▶1	123	Amy	3.9	1000
2	234	Bob	3.6	1500
3	345	Craig	3.5	500
4	456	Doris	3.9	1000
5	567	Edward	2.9	2000
6	678	Fay	3.8	200
7	789	Gary	3.4	800
8	987	Helen	3.7	800
9	876	Irene	3.9	400
10	765	Jay	2.9	1500
11	654	Amy	3.9	1000
12	543	Craig	3.4	2000

Students

	cName	state	enrollment
▶1	Stanford	CA	15000
2	Berkeley	CA	36000
3	MIT	MA	10000
4	Cornell	NY	21000

Colleges

	sID	cName	major	decision
▶1	123	Stanford	CS	Y
2	123	Stanford	EE	N
3	123	Berkeley	CS	Y
4	123	Cornell	EE	Y
5	234	Berkeley	biology	N
6	345	MIT	bioengineering	Y
7	345	Cornell	bioengineering	N
8	345	Cornell	CS	Y
9	345	Cornell	EE	N
10	678	Stanford	history	Y
11	987	Stanford	CS	Y
12	987	Berkeley	CS	Y
13	876	Stanford	CS	N
14	876	MIT	biology	Y
15	876	MIT	marine biology	N
16	765	Stanford	history	Y
17	765	Cornell	history	N
18	765	Cornell	psychology	Y
19	543	MIT	CS	N

Applies

Pros and Cons of Relational Models

- Pro:
  - Very flexible
  - Very expressive
  - Relational DBMS (RDBMS) can perform queries very fast
  - Other advantages …
- Con:
  - Quite complex to design
  - Quite complex to use
  - Requires a DBMS to manage tables, schemas, and queries
  - Other dis-advantages …

Relational Database-Search VS Web-search

Relational Database Queries

- Strict syntax, for example, SQL (Select …),
- Need tables to form queries
- Results = set of records
- Very precise results(exact results)
- Can manipulate data, for example, UPDATE

Web-search (for example, Google-search)

- For example, keyword-based, plain text, …
- No need database model to formulate queries
- Results = pages, pdfs, videos, etc.
- Not very precise results(for example, not exact and not sorted)
- Cannot manipulate data

More intelligent search such as semantic-based search …

Document Model

- Represents as semi-structured text documents
- Typically: Decide on a main entity type, and *embed all* its related sub-entities in a substructure

- Can be easily turned into a file, like JSON or XML
- Queries can be used to filter the document
- Often used in Web APIs and Web systems
- Also used for exchanging data

XML(eXtensible Markup Language) vs JSON (JavaScript Object Notation)

XML

```
<empinfo>
   <employees>
      <employee>
         <name>James Kirk</name>
         <age>40></age>
      </employee>
      <employee>
         <name>Jean-Luc Picard</name>
         <age>45</age>
      </employee>
      <employee>
         <name>Wesley Crusher</name>
         <age>27</age>
      </employee>
   </employees>
</empinfo>
```

JSON

```
{  "empinfo" :
      {
            "employees" :  [
            {
                  "name" : "James Kirk",
                  "age" : 40,
            },
            {
                  "name" : "Jean-Luc Picard",
                  "age" : 45,
            },
            {
                  "name" : "Wesley Crusher",
                  "age" : 27,
            }
```



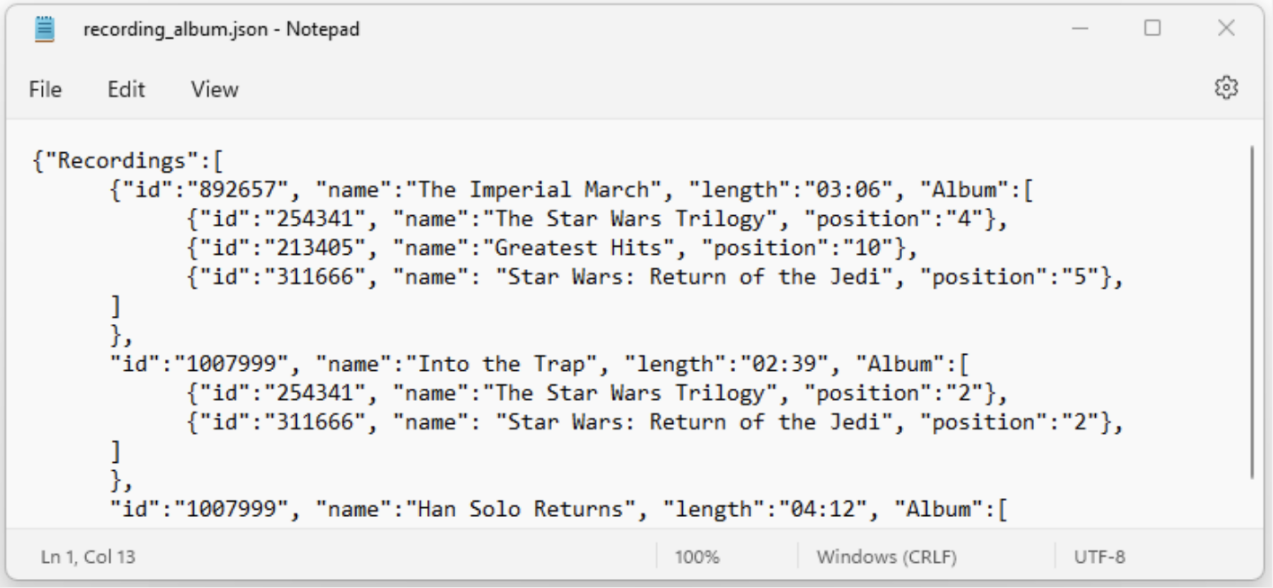
Music database using the document model

- **Logical Schemas:** represents all entity types and relationships to be stored in a database in the chosen data model
- **Example:** Example: A logical schema of our music database using the document model
- We need to choose which aggregation perspective to adapt!



Music database using the document model

- **Logical Schemas:** represents all entity types and relationships to be stored in a database in the chosen data model
- **Example:** Example: A logical schema of our music database using the document model: **JSON** (JavaScript Object Notation)



Option 3?

RecordAlbum
Record {
id
name
length
Album {
id
name
position
}
}

Pros and Cons of Document Model

- Pro:
- Simple
- **Dynamic Schema (Schema-less) =>** Evolve Without Downtime
- Can easily be converted to files
- Some structure remains (compared to single table model)
- No DBMS needed (but there are good Document DBMS)
- Con:
- Somewhat inflexible - Queries can be performed but they are typically less powerful than for relational model
- Some data redundancy depending on aggregation perspective

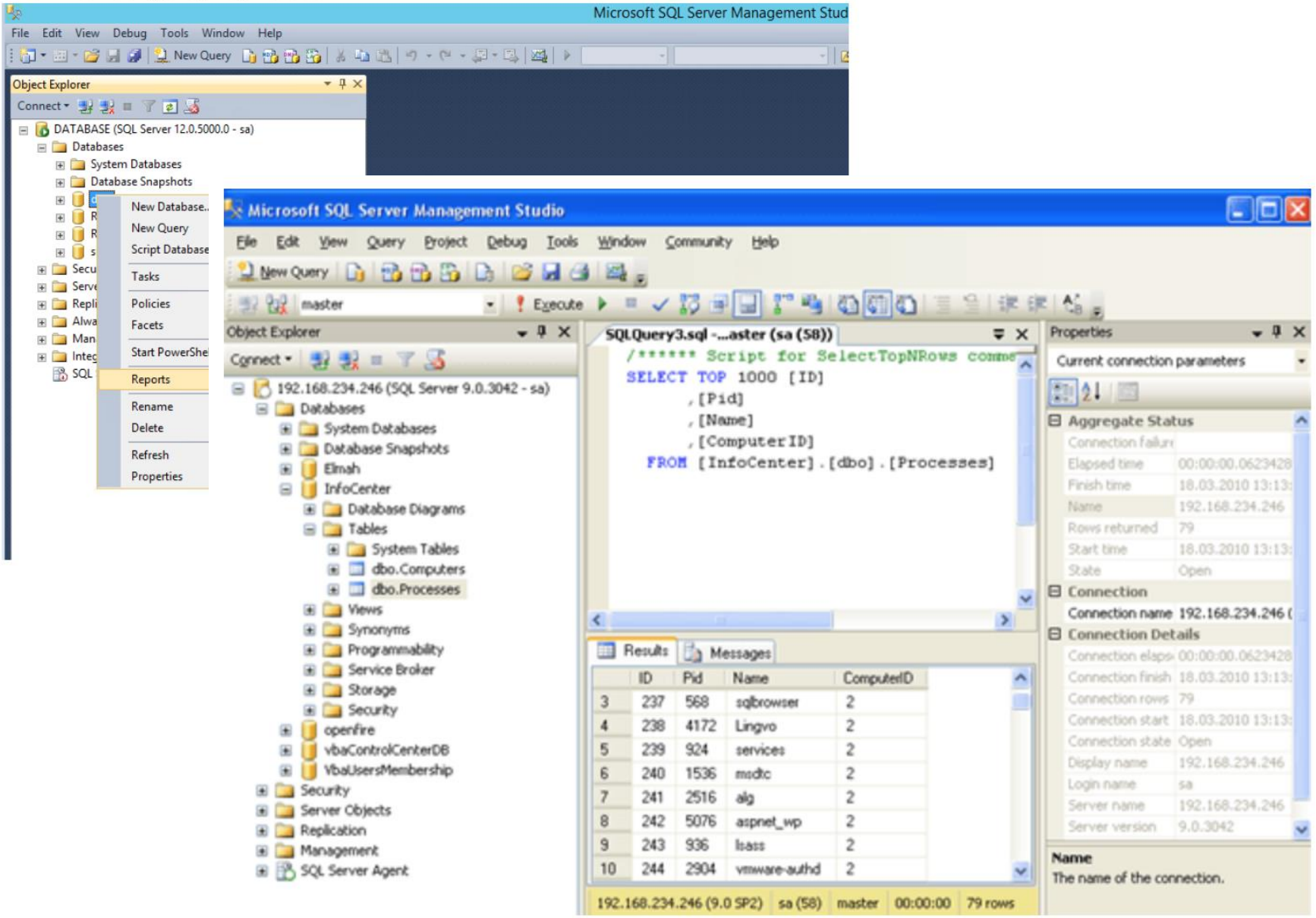
Some applications with DB models used

Applications	Database Model	Details
(some) **Adobe Systems** - graphic	- Relational model (SQLite)	File based storage in - Adobe Photoshop Lightroom - Adobe Reader
Forbes - business magazine	- Document model (MongoDB)	- Reconstruct its mobile application based on MongoDB to allow real-time content updates for viewers globally. - Reduce overhead
Walmart	- Graph model (Neo4j)	- Help understand customers' behavior and the relationship between products to enhance product recommendation. - Real-time data management.

Database Management System (DBMS)

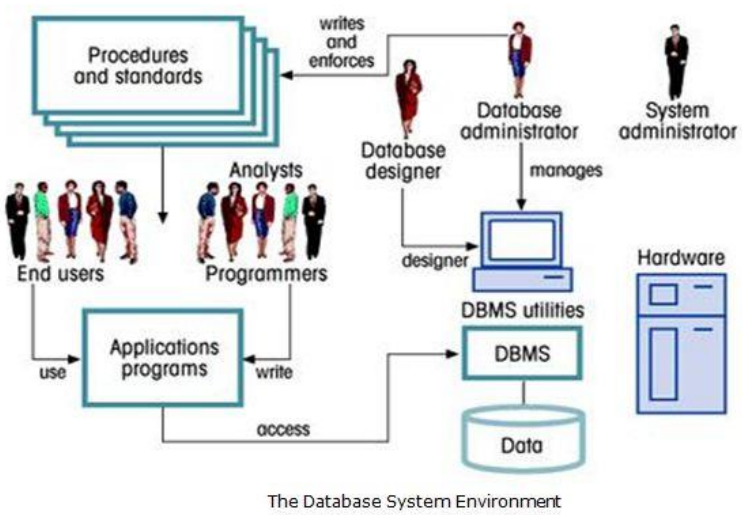
- **A Database Management System (DBMS)**
  - Software interface between the database and its end-users or applications
  - Software that stores, organizes, manages, and provides controlled access to data
  - DBMS main purpose is to allow users and applications to interact with data efficiently, securely, and reliably.
- **A DBMS contains**
  - Collection of interrelated data
  - Set of programs to access the data
  - An environment that is both *convenient* and *efficient* to use
  - GUI & Software development tools
- **A DBMS provides:**
  - Data storage and Data retrieval
  - Data update and manipulation
  - Security access , control, backup & recovery
  - Data consistency and integrity
  - Scalability and performance management

MS-SQL-Server: Example of R-DBMS



Main users of DBMS

- Database designers,
- Developers, Programmers
- Database Administrators
- System Analysts
- End-users (via Applications)
- …



The Database System Environment

DBMS Examples

- Relational DBMS (RDBMS)
  - **MySQL**, **PostgreSQL**, **SQL-Server**, Etc.
- Document DBMS
  - **MongoDB**, **CouchDB**
- Key-Value DBMS
  - **Redis**, **Riak**, **DynamoDB**
- Graph DBMS
  - **Neo4j**, **Amazon Neptune**
- Column-Family DBMS
  - **Cassandra**, **HBase**
- Time-Series DBMS
  - **InfluxDB**, **TimescaleDB**
- Vector DBMS
  - **Pinecone**, **Milvus**, **Weaviate**
- Multi-Model DBMS
  - **ArangoDB**, **Cosmos DB**
- …

Examples of Popular R-DBMS

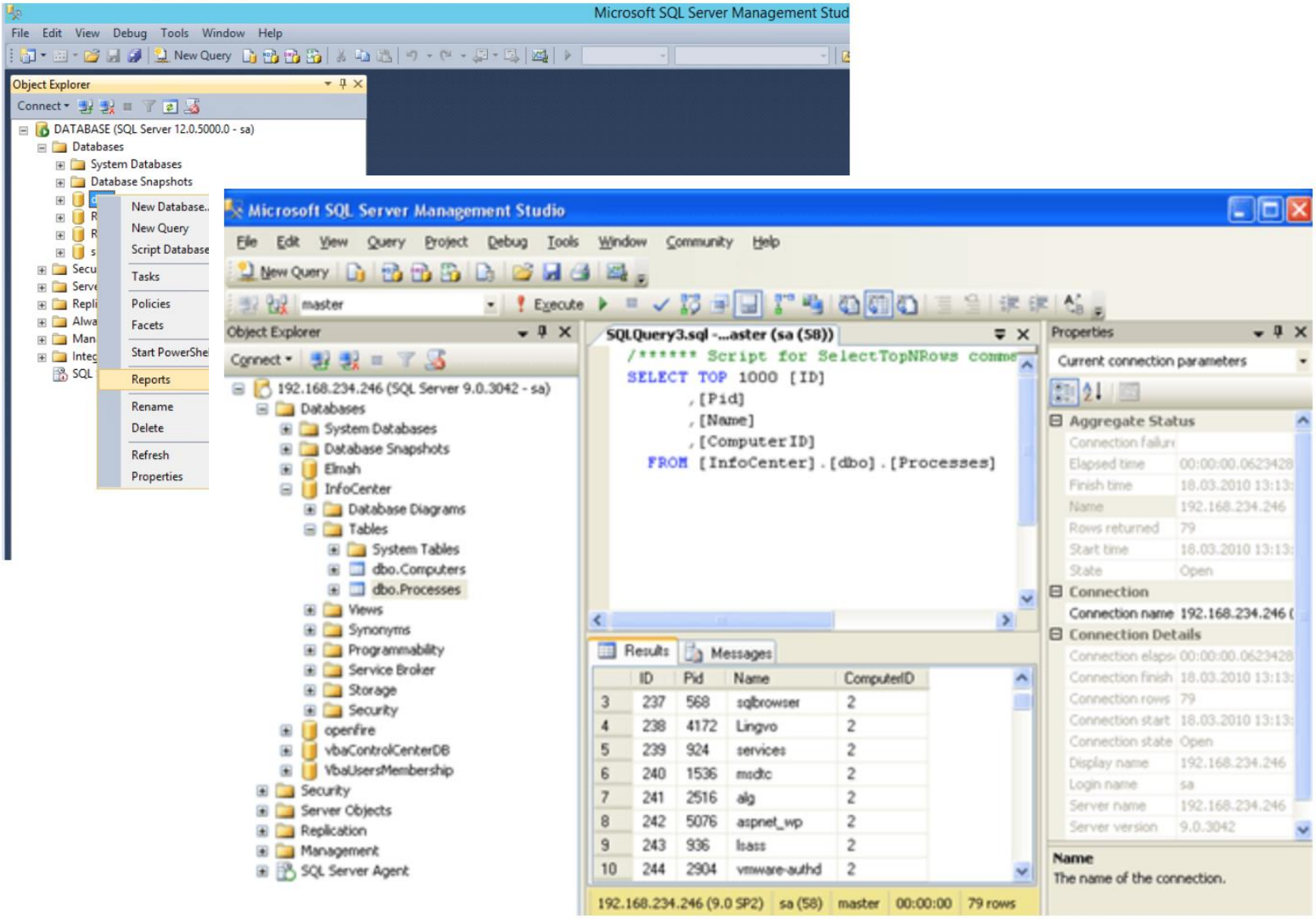
- Examples of popular RDBMSs:
  - Microsoft SQL-Server (Educational license)
  - IBM DB2
  - Oracle
  - **MySQL**
  - **PostgreSQL**
  - MS-Access
  - Etc.
- RDBMS provides
  - Efficient, reliable, convenient, and safe multi-user storage of and access to massive amounts of persistent data.

Main functions of R-DBMS

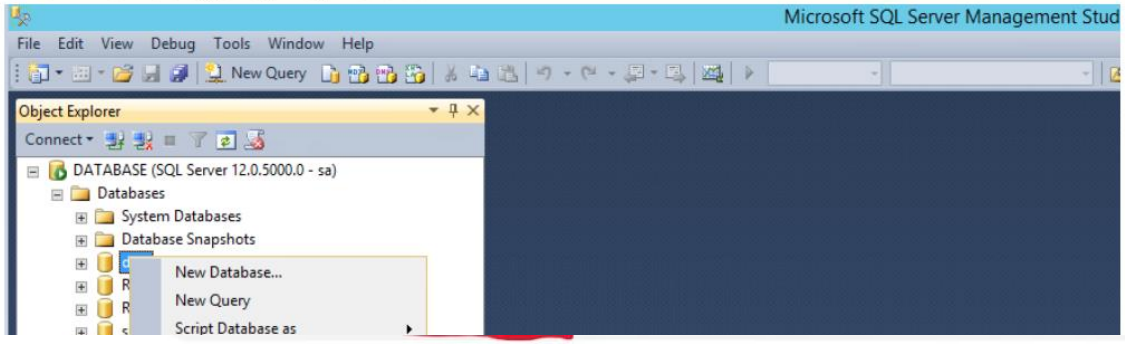
- Define and Store Database Structure
- Load Databases
- Store and Maintain Data
- Communicate with OS
- Security Control
- Backup & Recovery
- Concurrency Control
- Integrity Control
- Data Dictionary Management
- Etc.

- **Massive** => data not fit in memory, terabytes of data every day
- **Persistent** => program stops, data still there
- **Safe** => data not lost even hardware/software failure
- **Multi-user** => concurrency control
- **Convenient** => physical data independence, high-level query language
- **Efficient** => thousand of queries/updates per second
- **Reliable** => 99.99%

RDBMS example: MS-SQL-Server



RDBMS example: MS-SQL-Server



SQL Statement:

```
SELECT * FROM Customers;
```

Edit the SQL Statement, and click "Run SQL" to see the result.

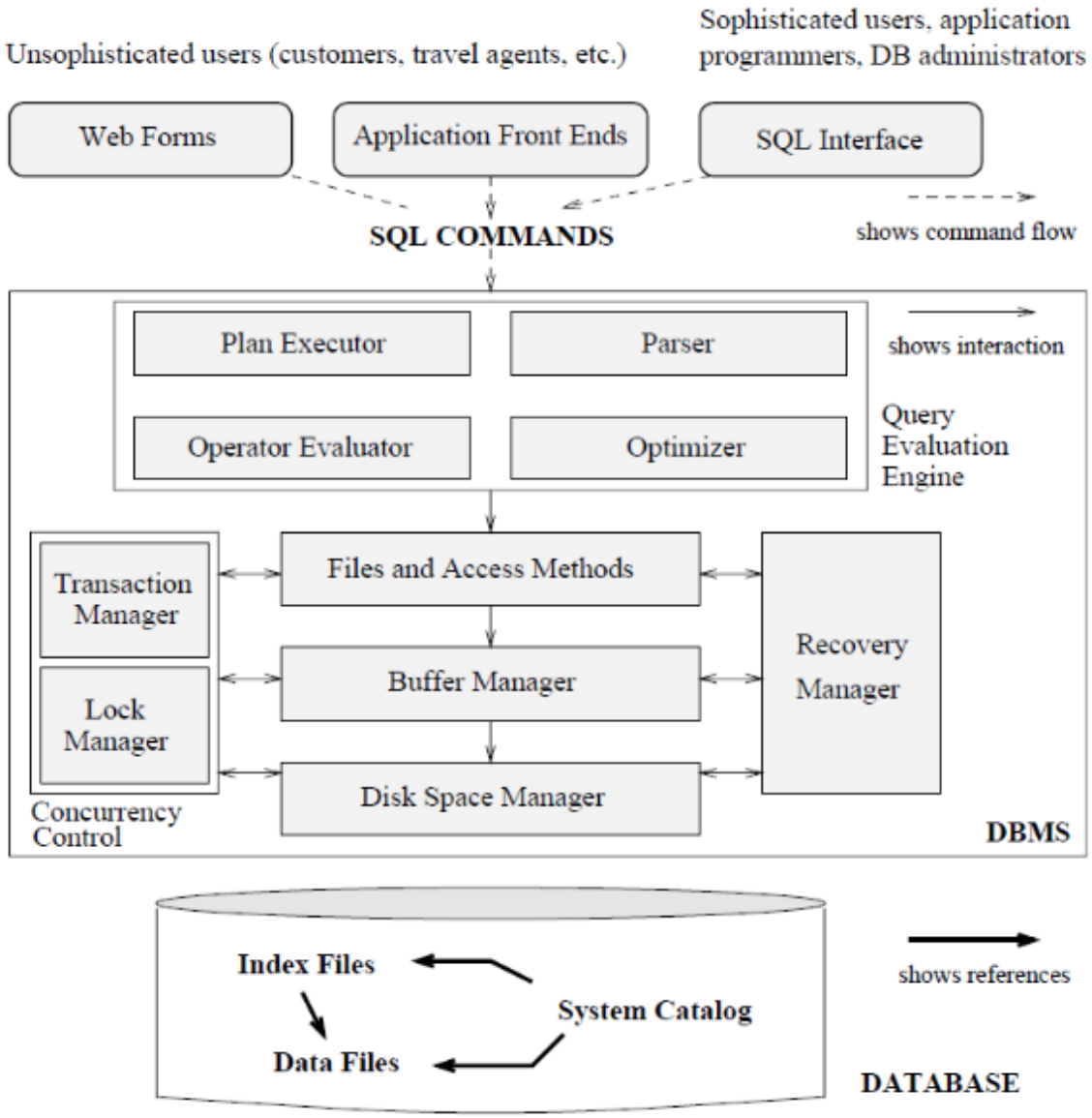
Run SQL »

Your Database:

Tablename	Records
Customers	91
Categories	8
Employees	10
OrderDetails	518
Orders	196
Products	77
Shippers	3
Suppliers	29

Restore Database

Architecture of R-DBMS



Popular Combination of Web-Programming & R-DBMS

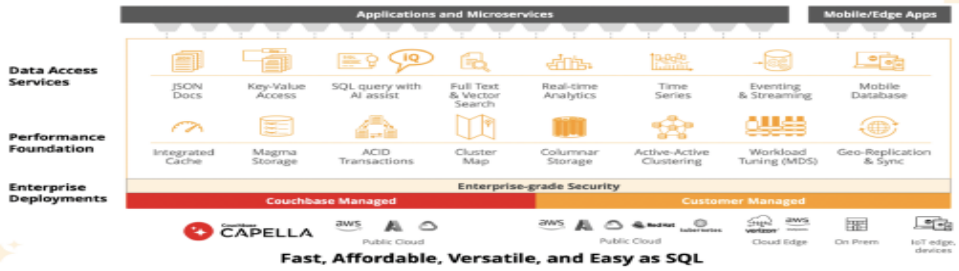


- **PHP/ASP and MySQL**
- VB and MS-Access
- VB and MS-SQL-Server
- C#.net Visual Studio and MS-SQL-Server
- Pyhton with MS-SQL-Server
- …

Database Models & Their DBMS

Database Models	DBMS
Relational Model	MySQL, SQL-server, PostgreSQL
Document Model	MongoDB, CouchDB
Column-Family Model	Cassandra, HBase
Key-Value Model	Redis, Riak
Graph Model	Neo4j, Amazon Neptune, FlockDB
Multi-Model	ArangoDB, OrientDB
NewSQL	CockroachDB, Google Spanner
Time-Series Model	InfluxDB, TimescaleDB

Examples of Modern DBMS

DBMS	Data Model	Characteristics / Components
1) PostGIS https://postgis.net/	Relational, Spatial, Multi-Model	- Extension of PostGreSQL - Stores non-spatial data and spatial data (lines, polygons, etc.) in 2D and 3D - Query data through its location and various functions - Tools for processing geometry - Geometry column - Raster data (grid-based: \=elevation and weather) - Topology Module - Geocoding / Reverse Geocoding - Supports cross-platform use (ArcGIS, Tableau, etc.)
2) Couchbase - NoSQL https://www.couchbase.com/	Document, Multi-Model	- Store data in RAM - Can be access from anywhere and can be used on cloud - Support large data processing application - Allows multiple data pattern to be access simultaneously **Couchbase's Differentiated Architecture** 
3) SQLite https://www.sqlite.org/	Relational	- C-language library - Stable file-based format (long-term archival) / cross-platform - Backwards compatible - Reduce latency - Acts as containers for transferring data - Can be used without an administrator

Some applications of databases with their DBMS and Database Models

- X
  - MySQL, PostgreSQL, Vertica - Relational DB (user authentication and structured data.)
  - Redis - NoSQL DB (real-time caching and session management)
  - Cassandra - NoSQL DB (storing tweets at scale)
  - FlockDB - Graph DB (store who follows whom and such)
- Instagram
  - PostgreSQL - Relational DB
  - Cassandra - NoSQL distributed database
  - Redis – NoSQL (store feed in real time)
- Netflix
  - Apache Cassandra and DynamoDB - NoSQL DB (manage video metadata, comments, and recommendations)
  - MySQL - Relational DB (metadata management)